ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

Excepciones

Laboratorio 5

Oscar Lasso

Juan Diego Gaitán

Profesor:

Maria Irma Diaz Rozo

Juan Frasica

Desarrollo Orientado a Programación de Objetos

Escuela Colombiana de Ingeniería Julio Garavito

Bogotá D.C.

ESCUELA COLOMBIANA DE INGENIERÍA
DESARROLLO ORIENTADO POR OBJETOS
Interfaz gráfica
2026-1 — Laboratorio 5/6

OBJETIVO

Evidenciar el logro de los siguientes resultados de aprendizaje:

1. Desarrollar una mini aplicación gráfica considerando el patrón MVC.
2. Implementar el esquema de manejo de eventos con clases anónimas
3. Experimentar el comportamiento de las ventanas `JFrame`, `JDialog` y `JOptionPane`
4. Seleccionar los lienzos más apropiados para un diseño: `JPanel` y `JTabbedPane`
5. Revisar las posibilidades de los estilos: `FlowLayout`, `BorderLayout` y `GridLayout`
6. Apropiar algunos componentes básicos: `JLabel`, `JTextField`, `JButton`, `JMenuBar`
7. Apropiar algunos componentes especiales: `JFileChooser` y `JColorChooser`
8. Experimentar las prácticas

   Testing Acceptance tests are run often and the score is published.
   Testing. When a bug is found tests are create.

ENTREGA

- Incluyan en un archivo `.zip` los archivos correspondientes al laboratorio. El nombre debe ser los apellidos de los dos miembros del equipo ordenados alfabéticamente y separados por un guion. (Casallas-Duarte)
- Publiquen el avance al final de la sesión y la versión definitiva en la fecha indicada, en los espacios correspondientes.

CONTEXTO

Reto: Implementar el juego **EasySokoban**

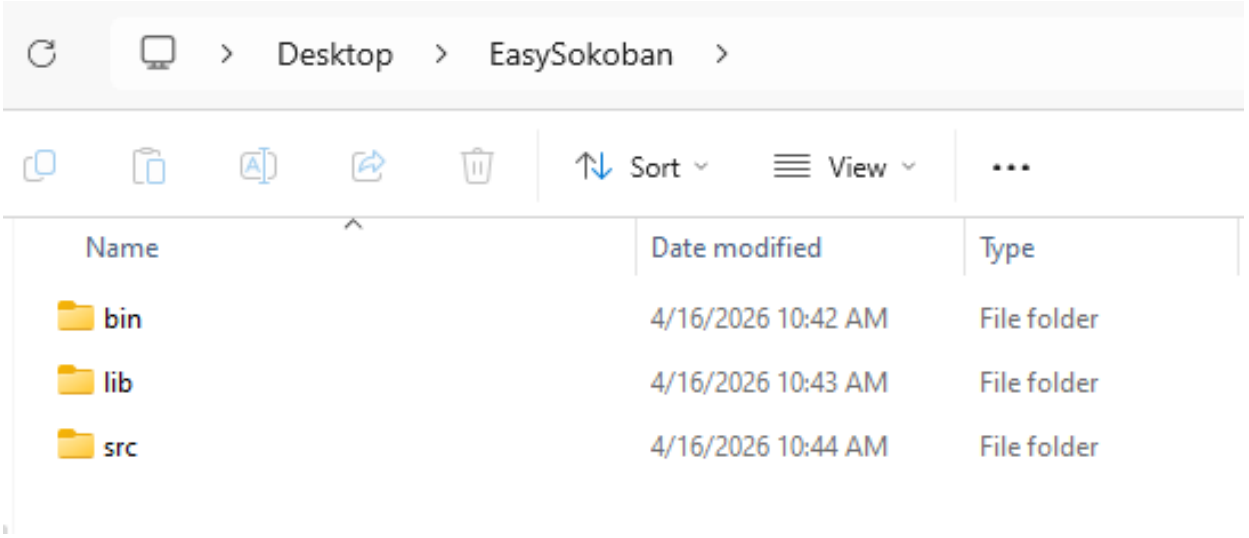
El propuesto por ustedes `SokobanGUI`	El acordado en laboratorio `Sokoban`
Vista-Controlador	**Modelo**

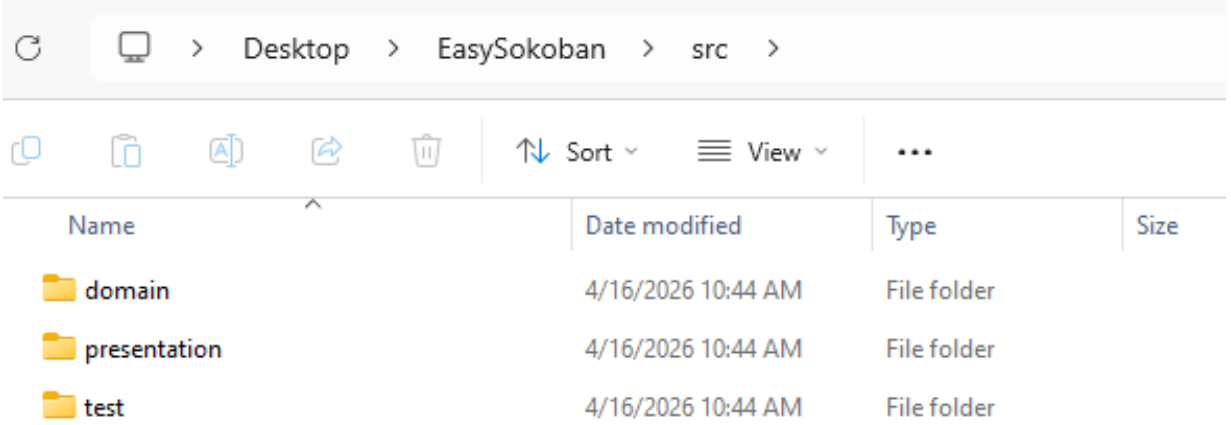
PARTE I. Maqueta

A Directorios [En lab05.doc]

El objetivo de este punto es preparar los directorios para el juego **EasySokoban**

1. Preparen un directorio llamado **EasySokoban** con los directorios `src`, `bin` y `lib`; y los subdirectorios para presentación, dominio y pruebas de unidad. Capturen una pantalla con la estructura.





B Ciclo 0: Ventana vacía – Salir [En lab05.doc *.java]

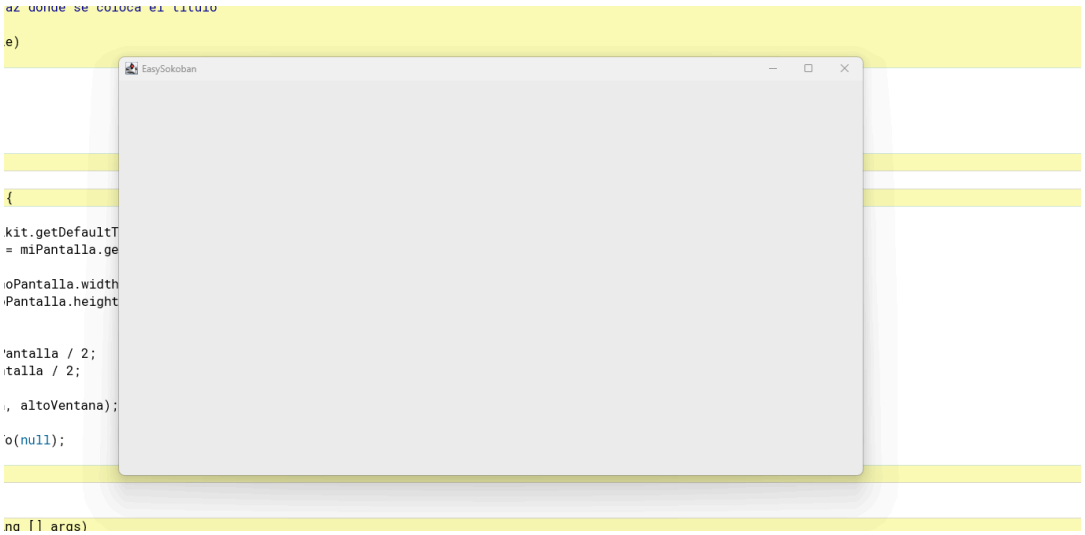
El objetivo es implementar la ventana principal de **EasySokoban** con un final adecuado desde el icono de cerrar. Utilizar el esquema de `prepareElements-prepareActions`.

1. Construyan el primer esquema de la ventana de **EasySokoban** únicamente con el título "**EasySokoban** ". Para esto creen la clase `SokobanGUI`como un `JFrame` con su creador (que sólo coloca el título) y el método `main` que crea un objeto `SokobanGUI`y lo hace visible. Ejecútenlo. Capturen la pantalla. (Si la ventana principal no es la inicial en su diseño, después deberán mover el `main` al componente visual correspondiente)

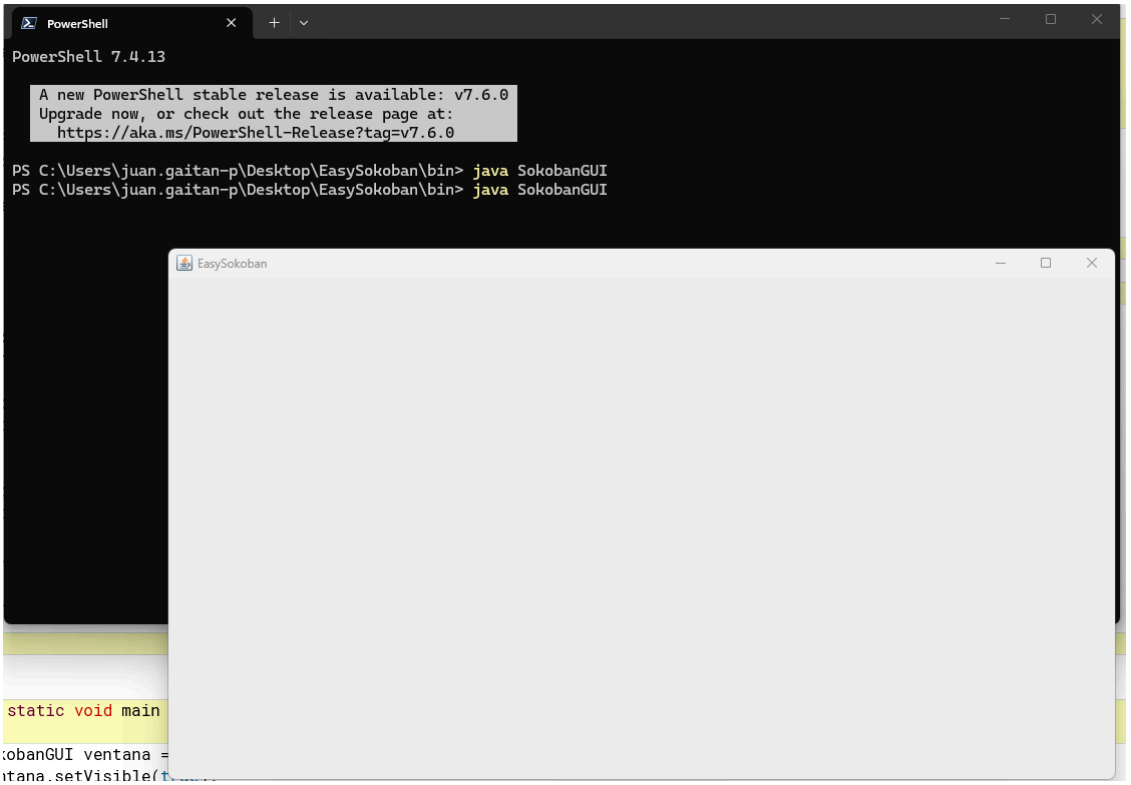
```java
import javax.swing.JFrame;


/**
 * Clase de GUI (Interfaz gráfica para el juego EasySkoban
 *
 * @author gaitan-lasso
 * @version 16/04/2026
 */
public class SokobanGUI extends JFrame{


    private String title;

    /**
     * Constructor para la interfáz donde se coloca el titul
     */
    public SokobanGUI(String title)
    {
        //Pasa el titulo
        super(title);

    }


    public static void main (String [] args)
    {
    SokobanGUI ventana = new SokobanGUI("EasySokoban");
        ventana.setVisible(true);
    }
}
```

2. Modifiquen el tamaño de la ventana para que ocupe un cuarto de la pantalla y ubíquenla en el centro. Para eso inicien la codificación del método `prepareElements`. Capturen esa pantalla.

```
public void prepareElements(){

    Toolkit miPantalla = Toolkit.getDefaultToolkit();
    Dimension tamanoPantalla = miPantalla.getScreenSize();

    int anchoPantalla = tamanoPantalla.width;
    int altoPantalla = tamanoPantalla.height;


    int anchoVentana = anchoPantalla / 2;
    int altoVentana = altoPantalla / 2;

    this.setSize(anchoVentana, altoVentana);

    this.setLocationRelativeTo(null);

}


public static void main (String [] args)
{
    SokobanGUI ventana = new SokobanGUI("EasySokoban");
    ventana.setVisible(true);
}
```



3. Traten de cerrar la ventana. ¿Termina la ejecución? ¿Qué deben hacer en consola para terminar la ejecución?



a) Cuando se cierra la ventana se quita la ejecución pero se bloquea el terminal.

b) Se debe usar el comando de salida forzosa CTRL + C para salir de ejecución.

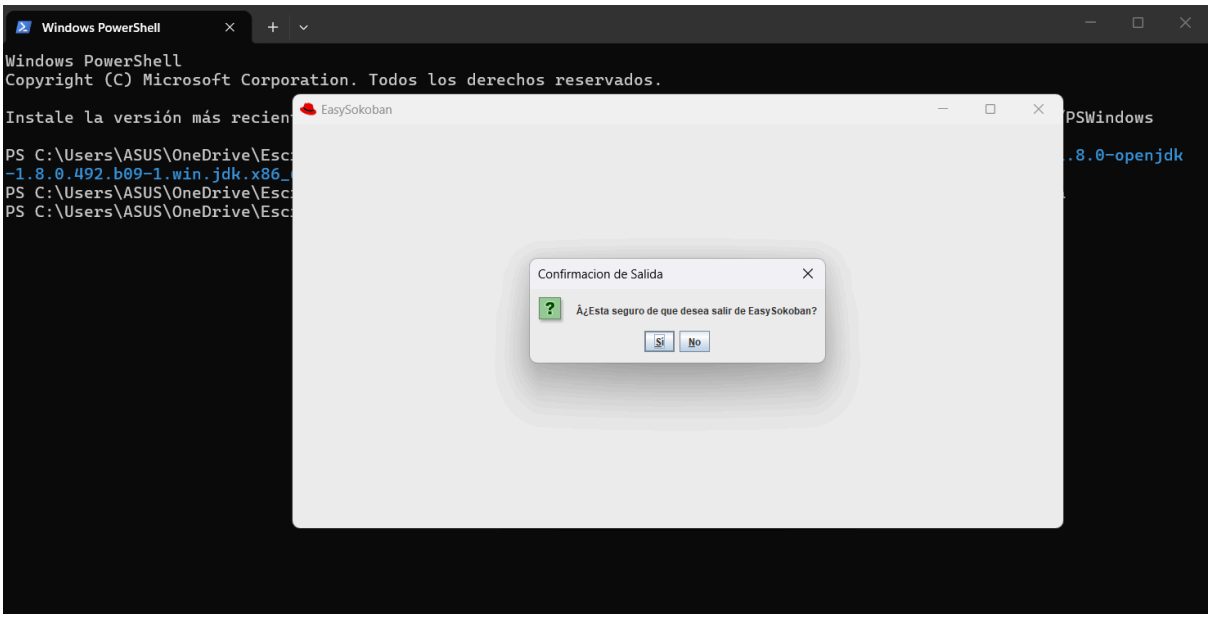
4. Estudien en `JFrame` el método `setDefaultCloseOperation`. ¿Para qué sirve? ¿Cómo lo usarían si queremos confirmar el cierre de la aplicación? ¿Cómo lo usarían si queremos simplemente cerrar la aplicación?

```
public void prepareElements(){

    Toolkit miPantalla = Toolkit.getDefaultToolkit();
    Dimension tamanoPantalla = miPantalla.getScreenSize();

    int anchoPantalla = tamanoPantalla.width;
    int altoPantalla = tamanoPantalla.height;


    int anchoVentana = anchoPantalla / 2;
    int altoVentana = altoPantalla / 2;

    this.setSize(anchoVentana, altoVentana);

    this.setLocationRelativeTo(c: null);

    this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);


}
```

a) Sirve para definir la acción de evento cuando se cierra la ventana JFrame.
b) Se utiliza EXIT_ON_CLOSE para decir que se deje de ejecutar todos los procesos en el terminal

5. Preparen el "oyente" correspondiente al icono cerrar que le pida al usuario que confirme su selección. Para eso inicien la codificación del método `prepareActions` y el método asociado a la acción (`exit`). Ejecuten el programa y cierren el programa. Capturen las pantallas.

```java
/**
 * Configura los oyentes de eventos de la ventana.
 */
public void prepareActions() {
    WindowListener oyenteCierre = new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            confirmarSalida();
        }
    };

    this.addWindowListener(oyenteCierre);
}

/**
 * Muestra un diálogo de confirmación antes de cerrar la aplicación.
 */
public void confirmarSalida() {
    int respuesta = JOptionPane.showConfirmDialog(
        this,
        "¿Esta seguro de que desea salir de EasySokoban?",
        "Confirmacion de Salida",
        JOptionPane.YES_NO_OPTION,
        JOptionPane.QUESTION_MESSAGE
    );

    if (respuesta == JOptionPane.YES_OPTION) {
        System.exit(0);
    }
}
```



C Ciclo 1: Ventana con menú – Salir [En lab05.doc *.java]

El objetivo es implementar un menú clásico para la aplicación con un final adecuado desde la opción del menú para salir. El menú debe ofrecer mínimo las siguientes opciones **Nuevo, Abrir – Salvar y Salir** . Incluyan los separadores de opciones.

1. Expliquen los componentes visuales necesarios para este menú. ¿Cuáles serían atributos y cuáles podrían ser varibles del método `prepareElements`? Justifique.

Para implementar un menú clásico en Java Swing, se requieren cuatro clases fundamentales del paquete javax.swing:

- JMenuBar: Barra horizontal principal que sostiene los menús.
- JMenu: Cada sección desplegable
- JMenuItem: Opciones clicables: Nuevo, Abrir, Salvar y Salir.
- JSeparator: Línea visual para agrupar opciones.

Atributos vs. Variables de Método

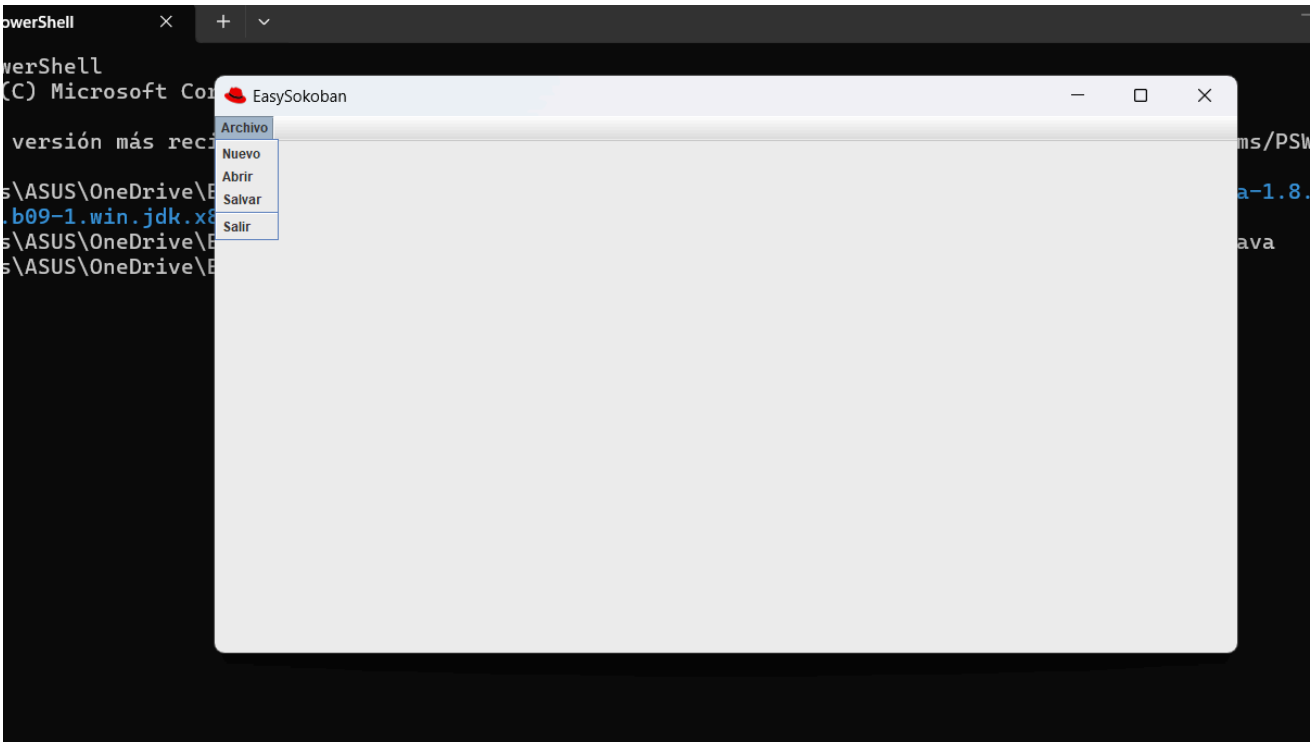
Para organizar el menú de EasySokoban, repartimos los componentes de la siguiente forma:

- **Atributos:** Declaramos como atributos los ítems del menú, es decir, Nuevo, Abrir, Salvar y Salir. Esto es necesario porque son los elementos que deben reaccionar cuando el usuario hace clic. Al ponerlos como atributos, permitimos que el método de las acciones pueda encontrarlos y asignarles su función sin problemas.
- **Variables de método:** La barra de menú y el menú principal de Archivo los dejamos como variables locales dentro del método donde se crea la interfaz. Como solo se necesitan para armar visualmente la ventana al principio y no vamos a interactuar con ellos después, no hace falta que ocupen espacio permanente en la memoria de la clase.

Esta división ayuda a que el código sea más ordenado. Solo dejamos con alcance global lo que realmente necesita ser controlado por eventos, manteniendo el resto del diseño protegido y ligero dentro de su propio método.

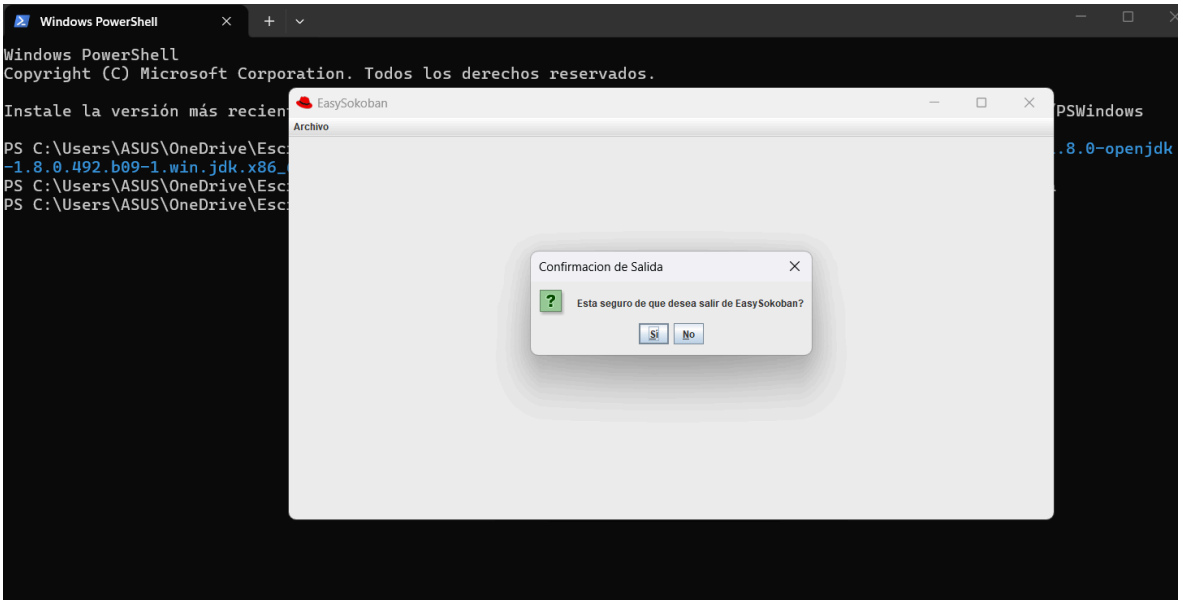
2. Construyan la forma del menú propuesto (`prepareElements` - `prepareElementsMenu`) . Ejecuten. Capturen la pantalla.

```java
/**
 * Construye y asigna la barra de menu a la ventana.
 */
private void prepareElementsMenu() {
    JMenuBar barraMenu = new JMenuBar();
    JMenu menuArchivo  = new JMenu("Archivo");

    itemNuevo  = new JMenuItem("Nuevo");
    itemAbrir  = new JMenuItem("Abrir");
    itemSalvar = new JMenuItem("Salvar");
    itemSalir  = new JMenuItem("Salir");

    menuArchivo.add(itemNuevo);
    menuArchivo.add(itemAbrir);
    menuArchivo.add(itemSalvar);
    menuArchivo.addSeparator();
    menuArchivo.add(itemSalir);

    barraMenu.add(menuArchivo);
    this.setJMenuBar(barraMenu);
}
```



3. Preparen el "oyente" correspondiente al icono cerrar con confirmación (`prepareActions` - `prepareActionsMenu`). Ejecuten el programa y salgan del programa. Capturen las pantallas.

```java
    menuArchivo.add(itemNuevo);
    menuArchivo.add(itemAbrir);
    menuArchivo.add(itemSalvar);
    menuArchivo.addSeparator();
    menuArchivo.add(itemSalir);

    barraMenu.add(menuArchivo);
    this.setJMenuBar(barraMenu);
}

/**
 * Configura los oyentes de eventos de la ventana.
 */
public void prepareActions() {
    WindowListener oyenteCierre = new WindowAdapter() {
        @Override
        public void windowClosing(WindowEvent e) {
            confirmarSalida();
        }
    };

    this.addWindowListener(oyenteCierre);
    prepareActionsMenu();
}
```

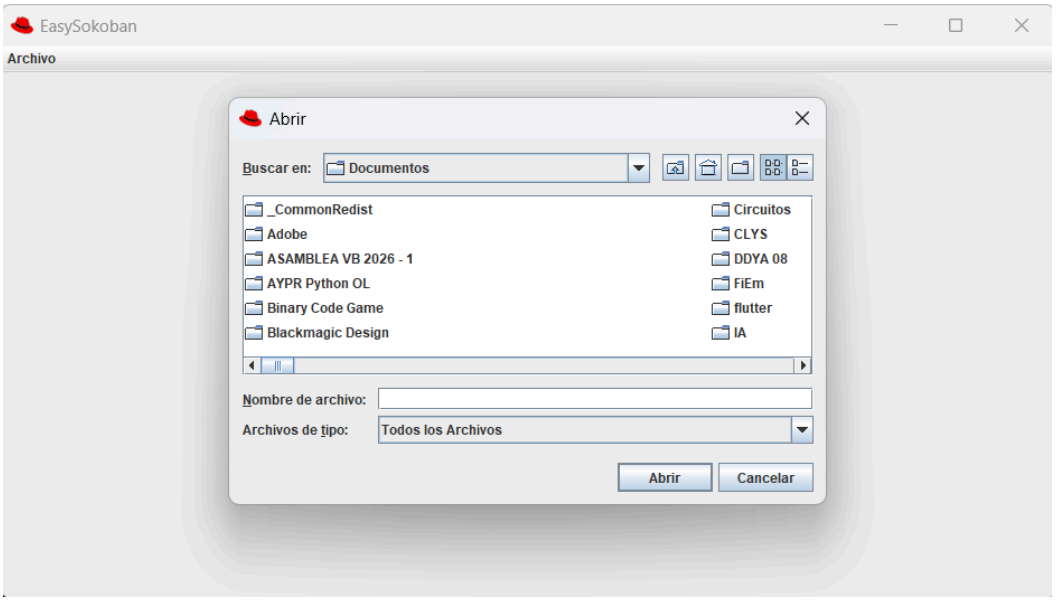


D Ciclo 2: Salvar y abrir [En lab05.doc *.java]

El objetivo es preparar la interfaz para las funciones de persistencia.

1. Detallen el componente `JFileChooser` especialmente los métodos : `JFileChooser`, `showOpenDialog`, `showSaveDialog`, `getSelectedFile`.
2. Implementen parcialmente los elementos necesarios para salvar y abrir. Al seleccionar los archivos indique que las funcionalidades están en construcción detallando la acción y el nombre del archivo seleccionado.
3. Ejecuten las dos opciones y capturen las pantallas más significativas.

```java
/**
 * Configura los oyentes de eventos del menu.
 */
private void prepareActionsMenu() {
    itemSalir.addActionListener(e -> confirmarSalida());

    itemAbrir.addActionListener(e -> {
        JFileChooser selector = new JFileChooser();
        int resultado = selector.showOpenDialog(this);
        if (resultado == JFileChooser.APPROVE_OPTION) {
            String nombre = selector.getSelectedFile().getName();
            JOptionPane.showMessageDialog(this,
                "Funcion de Abrir en construccion. Archivo: " + nombre);
        }
    });

    itemSalvar.addActionListener(e -> {
        JFileChooser selector = new JFileChooser();
        int resultado = selector.showSaveDialog(this);
        if (resultado == JFileChooser.APPROVE_OPTION) {
            String nombre = selector.getSelectedFile().getName();
            JOptionPane.showMessageDialog(this,
                "Funcion de Salvar en construccion. Archivo: " + nombre);
        }
    });
}
```



E Ciclo 3: Forma de la ventana principal [En lab05.doc *.java]

El objetivo es codificar el diseño de la ventana principal (todos los elementos de primer nivel)

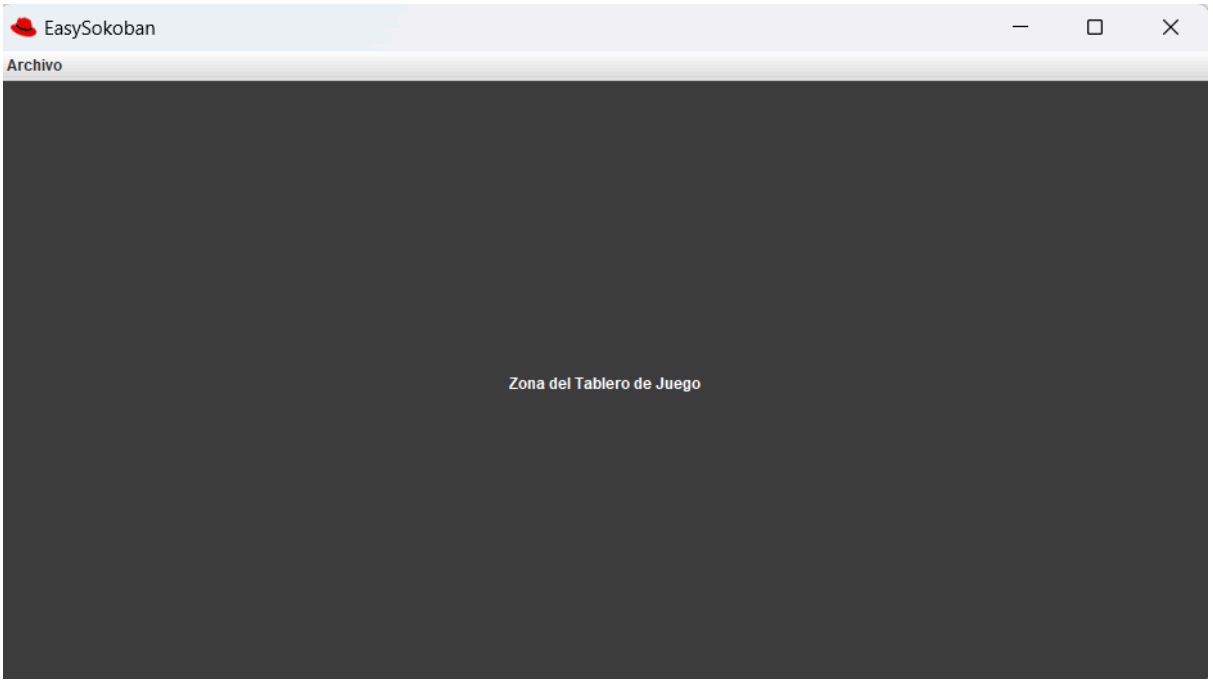
1. Presenten el bosquejo del diseño de interfaz con todos los componentes necesarios. (Incluyan la imagen)

```
+-------------------------------------------------------+
| Archivo (Nuevo, Abrir, Guardar, Salir)                |
+-------------------------------------------------------+
|                                                       |
|                                                       |
|                                                       |
|                [ TABLERO DE JUEGO ]                   |
|               (Grilla del Sokoban)                    |
|                                                       |
|                                                       |
|                                                       |
+-------------------------------------------------------+
| Nivel: 1    |    Movimientos: 12    | [ Reiniciar ]   |
+-------------------------------------------------------+
```

Se eligió un diseño basado en BorderLayout para separar claramente la barra de herramientas superior, el área interactiva del juego en el centro (que ocupará la mayor parte de la pantalla), y un panel de control en la parte inferior para mostrar el estado de la partida y botones de acción rápida. Esta separación facilita la implementación del método refresh() del tablero sin afectar la información de estado exterior.

2. Continúen con la implementación definiendo los atributos necesarios y extendiendo el método `prepareElements`. Para la zona del tablero definan un método `prepareElementsBoard` y un método `refresh` que actualiza la vista del tablero considerando, por ahora, el tablero inicial por omisión. Este método lo vamos a implementar realmente en otros ciclos.
3. Ejecuten y capturen la pantalla.

```java
/**
 * Inicializa el panel del tablero y lo agrega al centro de la ventana.
 */
private void prepareElementsBoard() {
    panelTablero = new JPanel(new BorderLayout());
    panelTablero.setBackground(Color.DARK_GRAY);

    JLabel etiqueta = new JLabel("Zona del Tablero de Juego", SwingConstants.CENTER);
    etiqueta.setForeground(Color.WHITE);
    panelTablero.add(etiqueta, BorderLayout.CENTER);

    this.getContentPane().add(panelTablero, BorderLayout.CENTER);
}

/**
 * Actualiza la vista del tablero.
 */
public void refresh() {
    // Aqui se dibujara el tablero inicial por omision en el futuro
    panelTablero.revalidate();
    panelTablero.repaint();
}
```



F Ciclo 4: Cambiar colores [En lab05.doc *.java]

El objetivo es implementar este caso de uso.

1. Expliquen los elementos (vista – controlador) necesarios para implementar este caso de uso.

En la parte de la vista, vamos a necesitar un componente visual que dispare la acción. Lo más común es agregar un nuevo JMenuItem en nuestra barra de menú que diga algo como "Cambiar Color". También hace parte de la vista la ventana emergente que le permite al usuario elegir el color, que en Java manejamos con la clase JColorChooser. Por último, el panelTablero que ya tenemos implementado será el componente visual que reflejará el cambio.

Por el lado del controlador, necesitamos crear un oyente de eventos, específicamente un ActionListener, y conectarlo al nuevo ítem de nuestro menú. La función de este controlador será muy específica: cuando el usuario haga clic en la opción, debemos ejecutar el método showDialog del JColorChooser para abrir la paleta. Si el usuario selecciona un color y acepta, el controlador captura ese color, se lo asignamos al panelTablero usando el método setBackground, y para terminar llamamos al método refresh que ya creamos para que la interfaz se actualice y muestre el nuevo color en pantalla.

2. Detalle el comportamiento de `JColorChooser` especialmente el método estático `showDialog`.

El JColorChooser es una ventana que ya viene lista en Java para que el usuario pueda elegir un color de una paleta.

La forma más práctica de usarlo es llamando a su método showDialog. Como es un método estático, se puede usar directamente sin tener que instanciar un objeto nuevo antes.

Al usar este método, pasan dos cosas importantes:

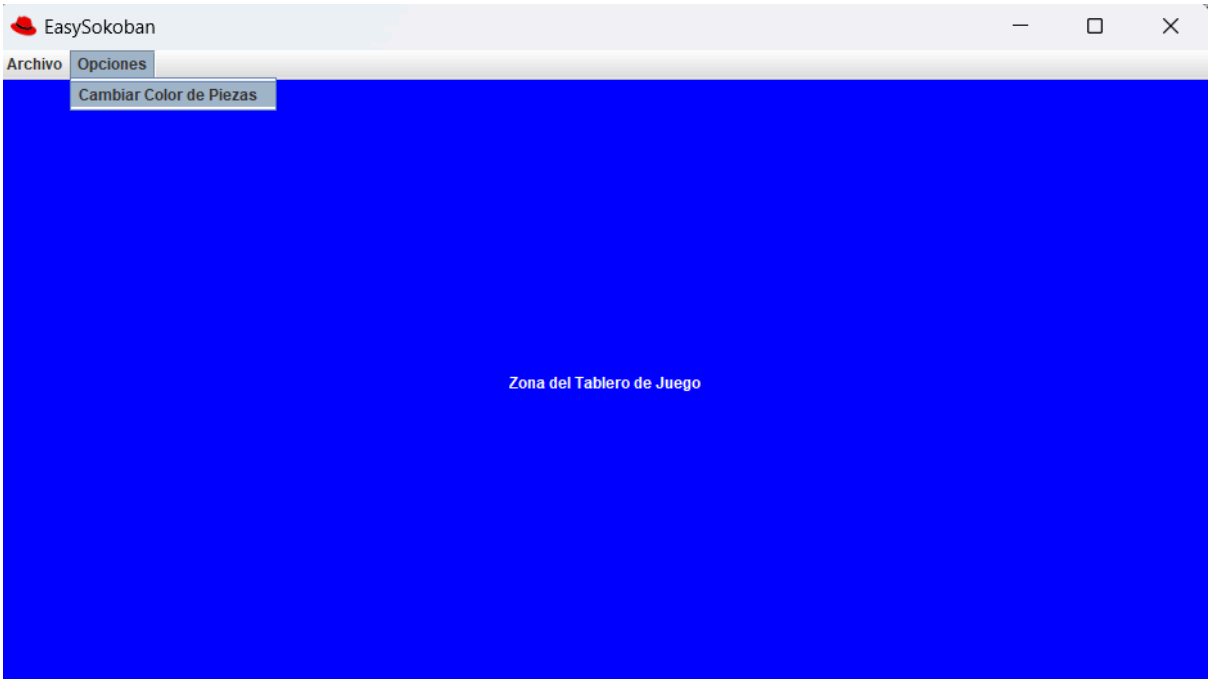
Primero, el juego se pausa temporalmente y se abre la paleta de colores por encima. Para que aparezca, el método nos pide tres parámetros básicos: sobre qué ventana queremos que se centre, el título del cuadro y el color que debe salir seleccionado por defecto.

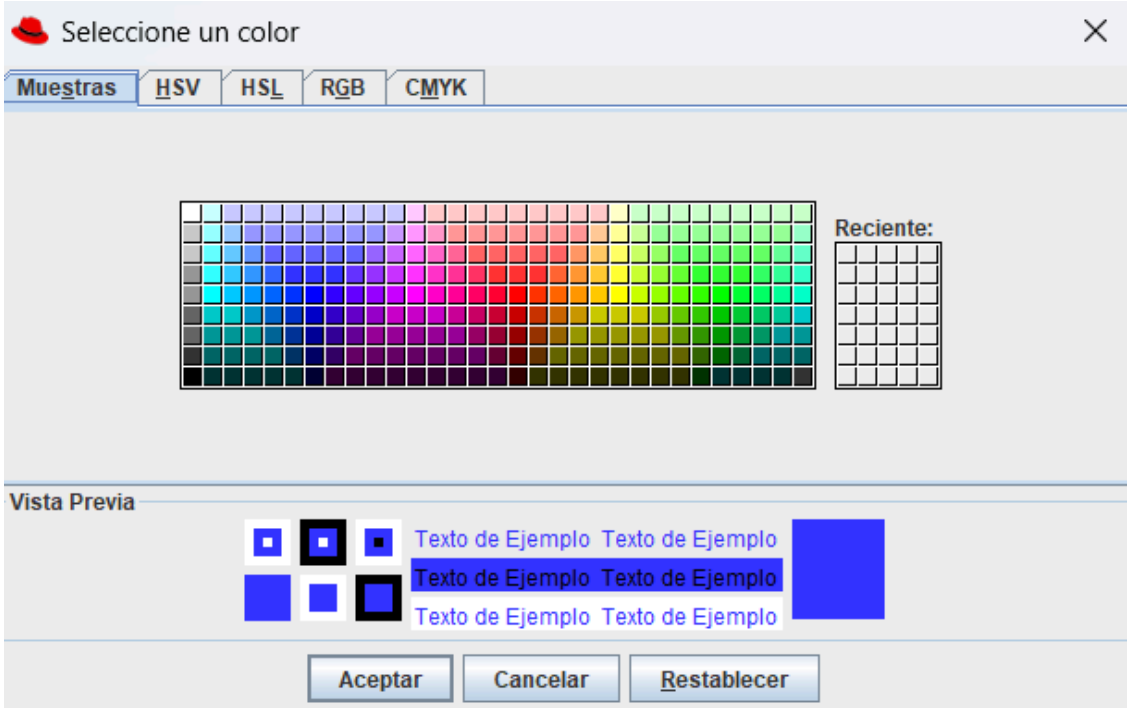
Segundo, cuando el usuario termina, el método nos da una respuesta. Si la persona elige un color y acepta, nos devuelve ese color para que lo usemos. Si decide cancelar o cerrar la ventana de golpe, nos devuelve un null. Por esto último, siempre hay que poner un condicional para verificar que el color no sea nulo antes de cambiar el fondo, y así evitar que el programa falle.

3. Implementen los componentes necesarios para cambiar el color de las piezas.
4. Ejecuten el caso de uso y capture las pantallas más significativas.

```
JMenu menuOpciones = new JMenu("Opciones");
itemColor = new JMenuItem("Cambiar Color de Piezas");
menuOpciones.add(itemColor);
```

```
    itemColor.addActionListener(e -> {
        Color seleccionado = JColorChooser.showDialog(
            this, "Seleccione un color", colorPiezas);
        if (seleccionado != null) {
            colorPiezas = seleccionado;
            refresh();
        }
    });
}
```





PARTE II. Funcionalidades básicas

A Ciclo 5: Modelo Sokoban [En lab05.doc easySokoban.astah *.java]

(NO OLVIDE BDD – MDD)

El objetivo es implementar la capa de dominio para **EasySokoban** .

1. Construya los métodos básicos del juego
2. Ejecuten las pruebas y capturen el resultado.

Para el desarrollo de esta fase, utilizamos como base el código que ya habíamos adelantado en la clase de TGUI con el proyecto Sokoban. A partir de allí, realizamos un proceso de refactorización, actualizando y modificando varios métodos para adaptarlos a la nueva estructura y requerimientos de este laboratorio

src > domain > CellType.java

```java
package domain;

/**
 * Tipos de casilla posibles en el tablero de EasySokoban.
 *
 * @author gaitan-lasso
 * @version 16/04/2026
 */
public enum CellType {
    FLOOR,
    WALL,
    GOAL
}

```

src > domain > Direction.java

```java
/**
 * @author gaitan-lasso
 * @version 16/04/2026
 */
public enum Direction {
    UP(-1, 0),
    DOWN(1, 0),
    LEFT(0, -1),
    RIGHT(0, 1);

    private final int dr;
    private final int dc;

    Direction(int dr, int dc) {
        this.dr = dr;
        this.dc = dc;
    }

    /**
     * Devuelve el desplazamiento en filas.
     *
     * @return delta de fila
     */
    public int dr() {
        return dr;
    }

    /**
     * Devuelve el desplazamiento en columnas.
     *
     * @return delta de columna
     */
    public int dc() {
        return dc;
    }
}
```

src > domain > EasySokoban.java

```java
/**
 *
 * @author gaitan-lasso
 * @version 16/04/2026
 */
public class EasySokoban {

    /** Numero de filas del tablero por omision. */
    public static final int DEFAULT_HEIGHT = 9;

    /** Numero de columnas del tablero por omision. */
    public static final int DEFAULT_WIDTH = 7;

    private static final double DEFAULT_DENSITY = 0.10;
    private static final int MIN_PLACEMENT_ATTEMPTS = 1000;
    private static final int ATTEMPTS_PER_ITEM = 20;

    private int height;
    private int width;
    private CellType[][] boardBase;

    private final Set<Position> boxes = new HashSet<>();
    private final Set<Position> goals = new HashSet<>();
    private Position player;
    private int moveCount;

    /**
     * Crea un tablero de 9 x 7 con generacion aleatoria al 10%.
     *
     * @return instancia lista para jugar
     */
    public static EasySokoban createDefault() {
        EasySokoban game = new EasySokoban();
        game.newBoard(DEFAULT_HEIGHT, DEFAULT_WIDTH);
        return game;
    }
```

src > domain > Position.java

```java
package domain;

import java.util.Objects;

/**
 * Representa una posicion inmutable en el tablero mediante fila y columna.
 *
 * @author gaitan-lasso
 * @version 16/04/2026
 */
public final class Position {

    private final int row;
    private final int col;

    /**
     * Crea una posicion con la fila y columna indicadas.
     *
     * @param row fila
     * @param col columna
     */
    public Position(int row, int col) {
        this.row = row;
        this.col = col;
    }

    /**
     * Devuelve la fila de esta posicion.
     *
     * @return fila
     */
    public int row() {
        return row;
    }
```

src > domain > EasySokobanTest.java

```java
package domain;

import org.junit.Before;
import org.junit.Test;

import static org.junit.Assert.*;

/**
 * Pruebas de comportamiento para la clase EasySokoban.
 * Convencion Given-When-Then (BDD).
 *
 * @author gaitan-lasso
 * @version 16/04/2026
 */
public class EasySokobanTest {

    private static final int HEIGHT = 9;
    private static final int WIDTH  = 7;

    private EasySokoban game;

    @Before
    public void setUp() {
        game = EasySokoban.createDefault();
    }
```

Windows PowerShell

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\ASUS\OneDrive\Escritorio\lab05\lab05-dopo-2026-02> $env:Path += ";C:\Users\ASUS\Downloads\java-1.8.0-openjdk
-1.8.0.492.b09-1.win.jdk.x86_64\java-1.8.0-openjdk-1.8.0.492.b09-1.win.jdk.x86_64\bin"
PS C:\Users\ASUS\OneDrive\Escritorio\lab05\lab05-dopo-2026-02> javac -d bin src/presentation/SokobanGUI.java
PS C:\Users\ASUS\OneDrive\Escritorio\lab05\lab05-dopo-2026-02> & "C:\Users\ASUS\.jdks\openjdk-26\bin\javac.exe" -encodin
g UTF-8 `
>>   -cp "lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" -d bin `
>>   src\domain\CellType.java src\domain\Direction.java `
>>   src\domain\Position.java src\domain\EasySokoban.java `
>>   src\domain\EasySokobanTest.java
PS C:\Users\ASUS\OneDrive\Escritorio\lab05\lab05-dopo-2026-02> & "C:\Users\ASUS\.jdks\openjdk-26\bin\java.exe" `
>>   -cp "bin;lib\junit-4.13.2.jar;lib\hamcrest-core-1.3.jar" `
>>   org.junit.runner.JUnitCore domain.EasySokobanTest
JUnit version 4.13.2
.....
Time: 0,027

OK (5 tests)

PS C:\Users\ASUS\OneDrive\Escritorio\lab05\lab05-dopo-2026-02>
```

B Ciclo 6: Jugar [En lab05.doc *.java]

El objetivo es implementar el juego

1. Expliquen los elementos necesarios para implementar este caso de uso.

Para lograr que la aplicación sea jugable, es necesario integrar la capa de dominio con la interfaz visual que ya construimos. Esto requiere la participación armónica entre la vista, el controlador y el modelo.

Por el lado del controlador, necesitamos implementar un oyente de teclado en la ventana principal. Su función será capturar las teclas que presione el usuario, traducirlas a las direcciones que entiende el juego y llamar al método encargado de mover al jugador. Justo después de cada movimiento, este controlador debe ordenarle a la vista que se actualice y revisar si ya se cumplió la condición para ganar el nivel.

En cuanto a la vista, nuestra clase de interfaz necesita guardar una referencia del modelo del juego para poder consultarlo. El cambio más importante aquí es modificar el panel del tablero para que se encargue de dibujar todo. Esto se hace sobrescribiendo su método de pintar, el cual tendrá que recorrer las dimensiones del juego y dibujar gráficamente dónde están las paredes, los destinos, las cajas y el jugador. A la par, usaremos etiquetas de texto en la parte inferior para mostrar estadísticas en tiempo real, como la cantidad de movimientos realizados.

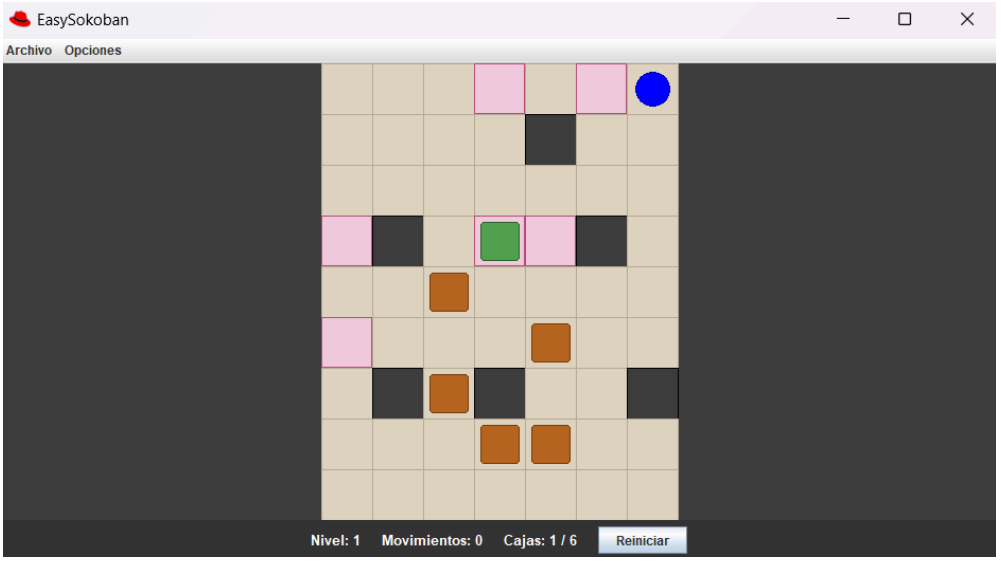
Finalmente, todo esto se apoya en el modelo de dominio. Aquí no hay que inventar funciones nuevas, sino reutilizar la lógica y los métodos que ya estructuramos en el ciclo anterior, aprovechando que la clase principal ya sabe cómo validar colisiones, empujar cajas y saber si el nivel está resuelto.

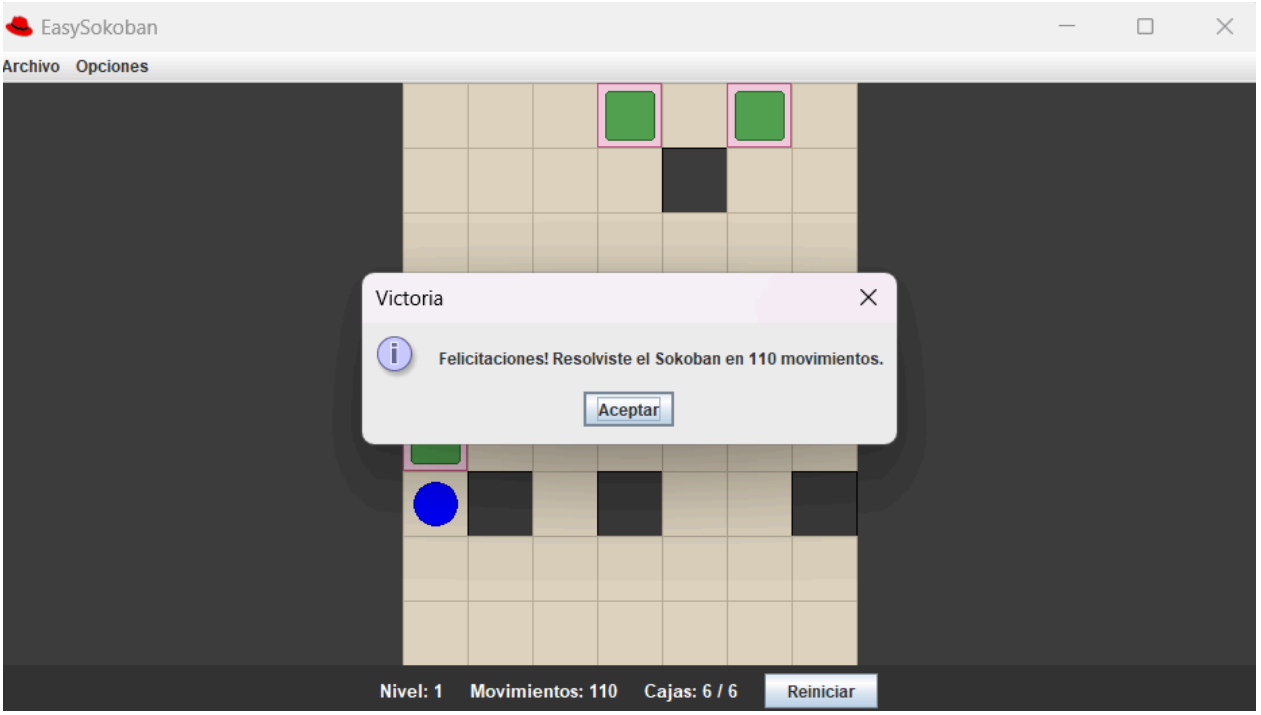
2. Implementen los componentes necesarios para jugar .¿Cuántos oyentes necesitan? ¿Por qué?
3. Ejecuten el caso de uso y capture las pantallas más significativas.

Para implementar el caso de uso principal de "Jugar", se necesita un solo oyente de eventos: un KeyListener.
Porque la única interacción en tiempo real requerida para jugar a EasySokoban es el movimiento del personaje en las cuatro direcciones básicas. El KeyListener es suficiente para capturar los eventos del teclado (las flechas direccionales o las teclas WASD), traducirlos a la capa de dominio y actualizar el modelo.

```java
public class SokobanGUI extends JFrame {

    /**
     * Dibuja el estado actual del tablero sobre el panel.
     *
     * @param g contexto grafico
     */
    private void dibujarTablero(Graphics g) {
        int filas    = juego.getHeight();
        int columnas = juego.getWidth();
        int ancho    = panelTablero.getWidth();
        int alto     = panelTablero.getHeight();

        int celdaAncho = ancho  / columnas;
        int celdaAlto  = alto   / filas;
        int tamCelda   = Math.min(celdaAncho, celdaAlto);

        int offsetX = (ancho  - tamCelda * columnas) / 2;
        int offsetY = (alto   - tamCelda * filas)    / 2;

        Position jugadorPos = juego.getPlayer();

        for (int fila = 0; fila < filas; fila++) {
            for (int col = 0; col < columnas; col++) {
                int x = offsetX + col * tamCelda;
                int y = offsetY + fila * tamCelda;

                Position pos = new Position(fila, col);
                CellType tipo = juego.cellTypeAt(pos);

                // Fondo de la celda
                if (tipo == CellType.WALL) {
                    g.setColor(new Color(60, 60, 60));
                    g.fillRect(x, y, tamCelda, tamCelda);
                    g.setColor(Color.BLACK);
                    g.drawRect(x, y, tamCelda, tamCelda);
                } else if (tipo == CellType.GOAL) {
```

```java
/**
 * Configura el oyente de teclado para mover al jugador.
 */
private void prepareActionsBoard() {
    this.addKeyListener(new KeyAdapter() {
        @Override
        public void keyPressed(KeyEvent e) {
            if (!enJuego) {
                return;
            }

            Direction dir = null;
            switch (e.getKeyCode()) {
                case KeyEvent.VK_UP:    dir = Direction.UP;    break;
                case KeyEvent.VK_DOWN:  dir = Direction.DOWN;  break;
                case KeyEvent.VK_LEFT:  dir = Direction.LEFT;  break;
                case KeyEvent.VK_RIGHT: dir = Direction.RIGHT; break;
                default: break;
            }

            if (dir != null) {
                boolean movio = juego.move(dir);
                if (movio) {
                    refresh();
                    verificarVictoria();
                }
            }
        }
    });

    this.requestFocusInWindow();
}
```





PARTE III. BONO: Funcionalidades complementarias

A Ciclo 7: Reiniciar [En lab05.doc *.java]

El objetivo es implementar este caso de uso.

1. Expliquen los elementos a usar para implementar este caso de uso.
2. Implementen los elementos necesarios para reiniciar
3. Ejecuten el caso de uso y capture las pantallas más significativas.

Para implementar el reinicio, se coordinan estos tres elementos:

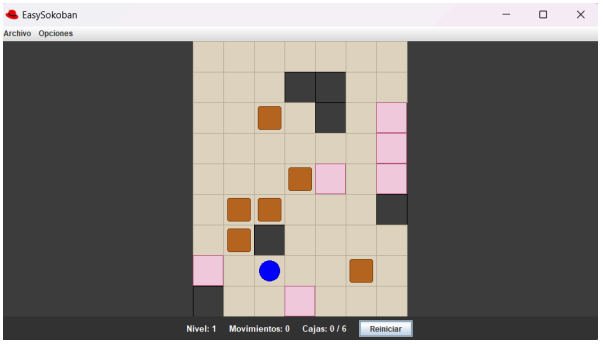
- Vista: Se usa el botón de "Reiniciar" (JButton) en la parte inferior como punto de interacción.

- Modelo: La clase del juego debe guardar las posiciones iniciales (jugador y cajas) y tener un método restart() que restablezca esas posiciones y limpie el contador de movimientos.

- Controlador: Un ActionListener en el botón que ejecute el reinicio del modelo, refresque la pantalla y use requestFocusInWindow() para que el teclado siga funcionando tras hacer clic.

```java
/**
 * Configura los oyentes de eventos del menu.
 */
private void prepareActionsMenu() {
    itemNuevo.addActionListener(e -> {
        nuevoJuego();
    });

    botonReiniciar.addActionListener(e -> {
        juego.restart();
        enJuego = true;
        refresh();
        this.requestFocusInWindow();
    });

    itemSalir.addActionListener(e -> confirmarSalida());

    itemAbrir.addActionListener(e -> {
        JFileChooser selector = new JFileChooser();
        int resultado = selector.showOpenDialog(this);
        if (resultado == JFileChooser.APPROVE_OPTION) {
            String nombre = selector.getSelectedFile().getName();
            JOptionPane.showMessageDialog(this,
                "Funcion de Abrir en construccion. Archivo: " + nombre);
        }
    });
```

→



B Ciclo 8: Cambiar el tamaño [En lab05.doc *.java]

El objetivo es implementar este caso de uso.

1. Expliquen los elementos a usar para implementar este caso de uso
2. Implementen los elementos necesarios para cambiar el tamaño del juego
3. Ejecuten el caso de uso y capture las pantallas más significativas.

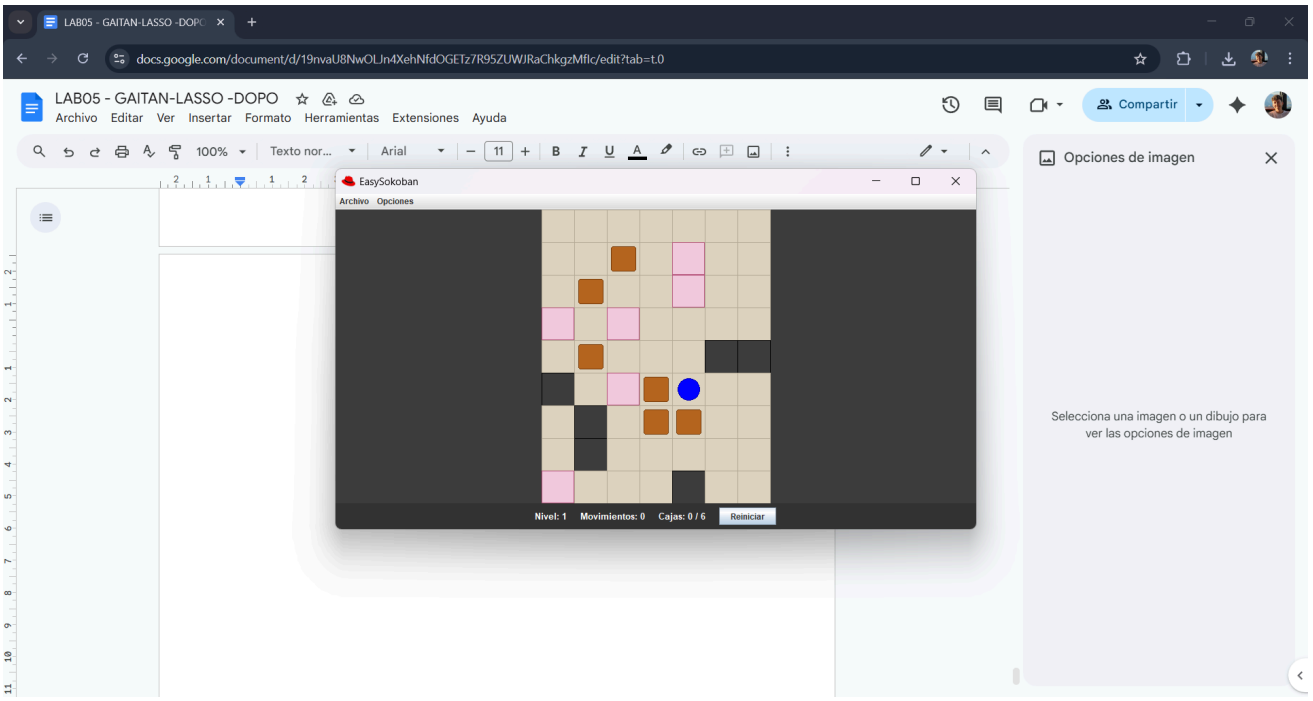
Para implementar el cambio de tamaño, se requieren estos elementos:

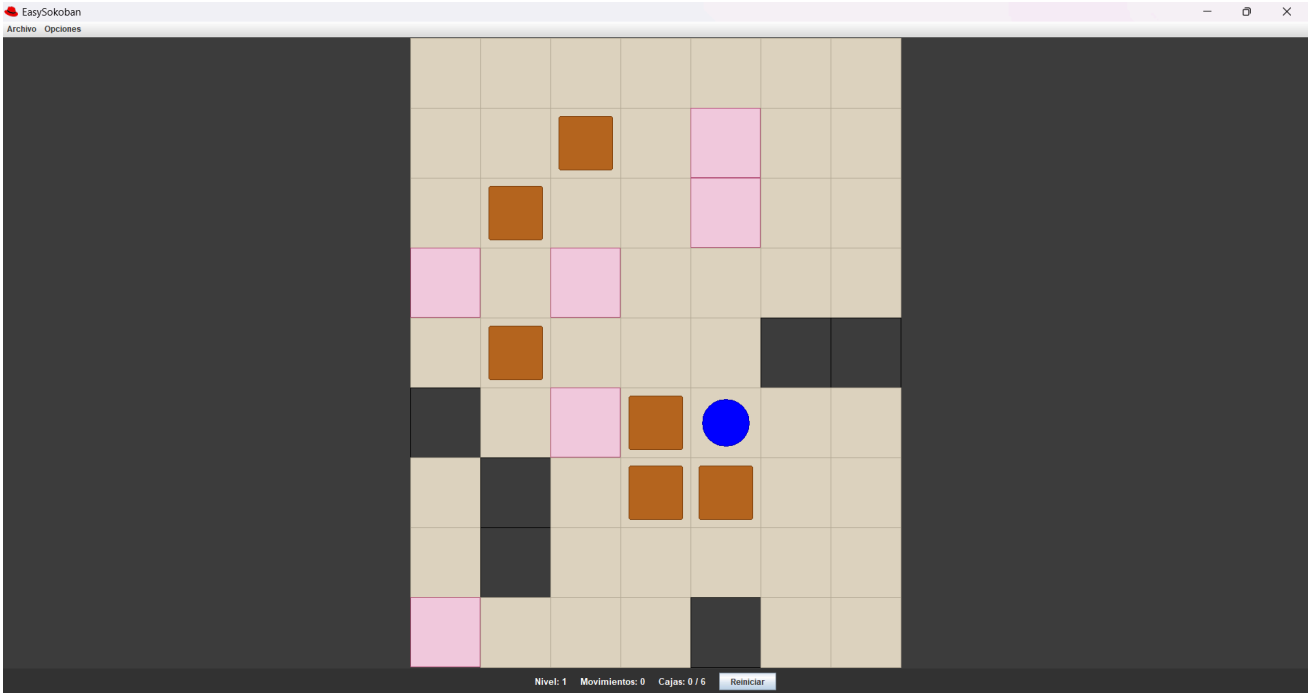
- Vista: Una opción en el menú "Opciones" que dispare un JOptionPane.showInputDialog para capturar las nuevas dimensiones (alto y ancho).

- Modelo: Un método en la clase del juego que reciba las nuevas medidas, limpie el tablero actual y genere uno nuevo calculando otra vez el 10% de elementos aleatorios (paredes, cajas y destinos) para el nuevo área.

- Controlador: Un ActionListener que valide que los datos ingresados sean numéricos y positivos, actualice el modelo y ejecute el método pack() del JFrame para que la ventana se redimensione automáticamente al nuevo tamaño del tablero.

```java
    itemDimension.addActionListener(e -> {
        String anchoStr = JOptionPane.showInputDialog(this, "Ingrese el ancho del tablero:");
        String altoStr = JOptionPane.showInputDialog(this, "Ingrese el alto del tablero:");

        try {
            if (anchoStr != null && altoStr != null) {
                int ancho = Integer.parseInt(anchoStr);
                int alto = Integer.parseInt(altoStr);
                if (ancho > 0 && alto > 0) {
                    juego.newBoard(alto, ancho); // EasySokoban usa (height, width)
                    enJuego = true;
                    this.pack();
                    refresh();
                    this.requestFocusInWindow();
                } else {
                    JOptionPane.showMessageDialog(this, "Las dimensiones deben ser positivas.");
                }
            }
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "Por favor ingrese números válidos.");
        }
    });
}
```


LAB05 - GAITAN-LASSO -DOPO
docs.google.com/document/d/19nvaU8NwOLJn4XehNfdOGETz7R95ZUWJRaChkgzMflc/edit?tab=t.0
LAB05 - GAITAN-LASSO -DOPO
Archivo Editar Ver Insertar Formato Herramientas Extensiones Ayuda
Compartir
Opciones de imagen
Selecciona una imagen o un dibujo para ver las opciones de imagen
EasySokoban
Archivo Opciones
Nivel: 1 Movimientos: 0 Cajas: 0 / 6
Reiniciar


EasySokoban
Archivo Opciones
Nivel: 1 Movimientos: 0 Cajas: 0 / 6
Reiniciar

RETROSPECTIVA

1. ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes?
18 horas en total (9 horas cada uno).

2. ¿Cuál es el estado actual del laboratorio? ¿Por qué?
Completo y funcional. Se implementaron todos los ciclos, logrando una separación clara entre la lógica del juego y la interfaz gráfica.

3. Considerando las prácticas XP del laboratorio, ¿cuál fue la más útil? ¿por qué?
Diseño Sencillo. Nos permitió avanzar rápido sin sobrecomplicar la lógica de movimiento y colisiones, facilitando la integración final.

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?
La generación aleatoria del tablero. Logramos que el sistema distribuya obstáculos y metas dinámicamente manteniendo siempre la proporción del 10% del área.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
La pérdida del foco del teclado. Al usar botones, las flechas dejaban de funcionar; lo solucionamos usando requestFocusInWindow() tras cada interacción.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
Mantuvimos una comunicación constante y equitativa. Nos comprometemos a aplicar más Pair Programming desde el inicio para detectar errores de lógica antes.

7. ¿Qué referencias usaron? ¿Cuál fue la más útil?

Oracle. Creating a GUI With JFC/Swing.
Beck, K. Extreme Programming Explained.